



Kanea Frontend

Architecture, Design Rationale & Defense Guide

A companion document to prepare for the project viva / defense

Component Covered: Frontend only (React + Vite customer app and admin panel)

Companion Files: `Kanea_Project_Report.html` ,
`Kanea_Backend_Design_Defense_Guide.html`

Purpose: Explain every frontend design decision and why it was made

Version: 2.0.0

Table of Contents

1. How to Use This Guide
2. The Frontend at a Glance
3. Design Philosophy — Why React + Vite, and Why No Router
4. Step-by-Step: How the Frontend Was Designed and Built
5. Deep Dive 1 — Real-Time Chat UI (ChatWindow.jsx)
6. Deep Dive 2 — Outage Board (OutageBoard.jsx)
7. Deep Dive 3 — Fault Report Wizard (FaultForm.jsx)
8. Deep Dive 4 — Admin Panel & Analytics Dashboard
9. Deep Dive 5 — App Shell, Routing & Network Configuration

- 10. Design Trade-offs — Alternatives Considered
- 11. Anticipated Defense Questions & Model Answers
- 12. Glossary of Key Terms

1. How to Use This Guide

This document gives you command of **why** the frontend was built the way it was — not just what each component renders. Section 4 is a chronological build procedure you can recite. Sections 5–9 are deep dives into each significant file with the reasoning behind specific implementation choices. Section 10 lists rejected alternatives. Section 11 rehearses the questions a panel is most likely to ask about the frontend, each with a tight model answer.

The one idea to hold onto

The frontend has **no build-time router, no global state library, and no chart library** — and that is a deliberate scope decision, not an oversight. The app has exactly one structural route split (customer app vs. admin panel), state that only ever needs to live in the component that uses it, and a handful of simple bar/histogram visualisations. Every dependency actually used (React, Vite, Tailwind, DOMPurify) earns its place; nothing was added "because it's standard."

2. The Frontend at a Glance

The frontend is a single React application built with Vite, split into two independent shells that share no state: the customer-facing app and the admin dashboard. Both are plain function components — there is no Redux/Context state management because no piece of state is needed by more than one component subtree.

frontend/

- └─ index.htmlVite entry HTML – mounts
- └─ vite.config.jsDev server + proxy config (0.0.0.0 host, /api and
- └─ tailwind.config.jsTailwind content scan paths
- └─ src/
 - └─ main.jsxEntry point – chooses App vs AdminPanel by URL pa
 - └─ config.jsAPI base path + dynamic WebSocket URL builder
 - └─ index.cssTailwind directives + custom scrollbar
 - └─ App.jsxCustomer shell: header, 3-tab nav, footer
 - └─ components/
 - └─ ChatWindow.jsxWebSocket chat UI, ratings, feedback, escalation
 - └─ OutageBoard.jsxLive outage list – polling, filtering, summary ca
 - └─ FaultForm.jsx3-step fault report wizard
 - └─ AdminPanel.jsxLogin gate + admin shell (4 tabs) + adminFetch()
 - └─ Analytics.jsxKPI cards, histograms, and bar charts (hand-built

File	Responsibility
ChatWindow.jsx	Owns the WebSocket connection; renders messages, quick re
OutageBoard.jsx	Displays live/scheduled/resolved outages with search and
FaultForm.jsx	Guided 3-step form for submitting an electrical fault rep
AdminPanel.jsx	Password gate issuing a token, then a 4-tab operations da
Analytics.jsx	Renders KPI cards and hand-built charts from aggregated m

Rendered Component Tree

```
main.jsx
└─ (URL path check: starts with "/admin"?)
   └─ NO → App.jsx
      │   └─ ChatWindow.jsx      (tab: Chat Support)
      │   └─ OutageBoard.jsx     (tab: Outage Status)
      │   └─ FaultForm.jsx       (tab: Report a Fault)
   └─ YES → AdminPanel.jsx
      └─ LoginGate               (shown until a token exists in sessi
      └─ AdminShell
         └─ Analytics.jsx        (tab: Analytics)
         └─ FaultReportsTab      (tab: Fault Reports)
         └─ EscalationsTab       (tab: Escalations)
         └─ RagSourcesTab        (tab: RAG Sources)
```

3. Design Philosophy – Why React + Vite, and Why I

Three frontend stacks were actually compared during Phase 1 design (see the project comparison from the presentation deck), and the same reasoning still holds for t

Factor	Vite + React (chosen)
Speed	Very fast dev server (native ES modules, no bundling)
Scalability	Highly scalable – component-based, easy to add tabs,
Cost	Free
Usability for the team	Easy to learn, flexible, rapid development

Within React, the decision to **not add** `react-router-dom` was made because the app only has one meaningful route split: customer app vs. admin panel. That split the earliest possible point, by reading `window.location.pathname` directly in `main.jsx`:

```
const isAdmin = window.location.pathname.startsWith('/admin')

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    {isAdmin ? <AdminPanel /> : <App />}
  </React.StrictMode>
)
```

Everything else – switching between "Chat Support", "Outage Status" and "Report four admin tabs – is not real navigation at all. It's a single `useState` variable (`tab`) controlling which child component renders, inside one already-mounted parent library exists to manage browser history, deep-linkable URLs, and nested route transitions app needs, since none of its internal tabs need their own shareable URL or back-

Why this matters for a viva

If asked "why no React Router?", the honest and correct answer is to **distinct routes to justify one** – adding a routing library to manage `(isAdminRoute ? <AdminPanel/> : <App/>)` would be exactly the kind of abstraction good engineering avoids. If the project grew to need shared individual chat sessions or fault report references, that would be justified.

4. Step-by-Step: How the Frontend Was Designed and

This is the procedure to recite when asked "walk me through how you built the inte

1

Scaffold the app shell around the four customer needs, not around

Before any component was styled, the four customer needs identified (chat, outage status, fault reporting, escalation) were mapped plus one always-visible action (the 0800-POWER call button in structure is a direct reflection of the problem statement, not

2

Set up Vite with Tailwind, and configure the dev-server proxy im

Vite was chosen for near-instant dev server startup and hot reload. up front so every component from the first commit onward used no separate CSS files to keep in sync per component. The proxy backend REST, `/ws` → backend WebSocket) were set up in `vite.c` before writing any component, so every component could assume what host the page was loaded from.

3

Build the chat UI against the WebSocket contract first

`ChatWindow.jsx` was built first because it is the most complex piece of state: an open socket, a running message list, and a typing indicator. The connect/reconnect/send/receive lifecycle right early meant even feedback, escalation banners) could be added as an incremental message loop instead of being designed alongside connection-handling.

4

Add the outage board and fault-report form as independent, self-contained components

Both `OutageBoard.jsx` and `FaultForm.jsx` were deliberately built to own their own state and fetch their own data (no props from a parent). This is a tab that can be developed, tested, and reasoned about in isolation.

5

Wire up feedback loops (ratings, thumbs up/down) once the chat was working

The 1-5 star session rating and per-message helpful/unhelpful buttons were added once the core chat worked, because they depend on the backend already running. The `message_id` per bot reply – a dependency that only existed once the write-path (Section 8 of the backend guide) was finished.

6

Build the admin panel as a second, independent React root – not a

Rather than nesting an admin view inside the customer `App` component, the admin panel was built as an entirely separate React root. A `main` component (`AdminPanel.jsx`) selected at the earliest possible point (`main.jsx`). This keeps admin-only code (fault report queue, escalations, a regular customer's browser reasons about admin-only UI element flashing on screen before an auth check

7

Move admin auth from a hardcoded frontend password to a backend-i

The earliest version checked the password in frontend code. Once backend authentication was added, `LoginGate` was rewritten to POST the password once to `/admin/login`, store only the returned opaque token in `sessionStorage`, and every subsequent admin call was routed through an `adminFetch()` wrapper that automatically attaches `Authorization: Bearer <token>`. This is the frontend half of the hardening described in the backend guide.

8

Build the analytics dashboard by hand instead of importing a charting library

Only three visualisation shapes were actually needed: a KPI stat card (intent distribution), and a vertical histogram (1-5 star ratings) as small components using plain Tailwind width/height percentages and `style` – avoiding a charting library dependency for a handful

9

Harden the rendering of AI-generated content against XSS

Once bot replies were being rendered with `dangerouslySetInnerHTML` (to show **bold** emphasis and multi-line formatting), every bot message was sanitized with `DOMPurify.sanitize()` with an explicit allow-list of safe tags (`strong`, `em`, `br`, `ul`, `li`, `p`) before being rendered – closing the XSS risk inherent in rendering user input as HTML.

10

Make the app reachable from any device on the network

`vite.config.js` was set to `host: true` (bind to `0.0.0.0` instead of just `localhost`), and `config.js`'s `WS_URL` was written to derive the WebSocket host from `window.location.host` at runtime rather than a hardcoded `localhost:8000`. Together, these two changes are what let a phone on the same Wi-Fi load the app by IP address and have chat still work without configuration change.

5. Deep Dive 1 – Real-Time Chat UI (ChatWindow.js)

Session identity is a client-generated random ID, not a login

```
function genSessionId() {  
  return 'sess-' + Math.random().toString(36).slice(2, 10)  
}
```

Because Kanea is anonymous customer support (no account system), a session needs the duration of one browser tab's conversation – enough to group messages, rating together in the database. The ID is generated once via `useState(genSessionId)` initializer form, so it only runs on first render) and reused for the life of the

WebSocket lifecycle: connect, reconnect with backoff, clean up

```
ws.onclose = () => {  
  setConnected(false)  
  // Backoff: 1s → 2s → 4s → 8s ... capped at 30s  
  const delay = Math.min(1000 * Math.pow(2, retryCount.current), 30_000)  
  retryCount.current += 1  
  retryTimeout.current = setTimeout(connect, delay)  
}
```

Exponential backoff exists so a dropped connection doesn't hammer the backend with every second – each failure waits longer than the last, up to a 30-second ceiling. `retryCount` resets to zero the moment a connection succeeds (in `onopen`). The `useEffect` cleanup function clears any pending retry timeout and closes the socket on unmount, preventing a reconnect loop from continuing after the component is gone.

Rendering AI output safely

```
function sanitize(text) {
  const html = formatISOdates(text).replace(/\/\*\*(.*?)\*\//g, '<strong>$1</strong>');
  return DOMPurify.sanitize(html, { ALLOWED_TAGS: ['strong', 'em', 'br', 'ul', 'li'] });
}
```

Two transforms happen before sanitisation: raw ISO timestamps embedded in bot replies (e.g. outage data) are converted to human-readable local date/time strings via `formatISOdates` and Markdown-style `bold` emphasis from the LLM's output is converted to `<strong>` tags. Only then is the result run through DOMPurify with an explicit allow-list – this ordering matters: sanitisation must be the *last* step, so nothing earlier transforms can reintroduce unsafe markup.

Why the rating prompt appears after exactly 5 bot replies

`addMessage()` increments a `botMsgCount` counter on every bot message and flips `showRating` to true the instant the count reaches 5. This is a deliberate UX choice to appear early (e.g. after 1 message) and the user hasn't had enough interaction to judge the bot yet (only at disconnect) and many users would never see the prompt at all because the

Per-message feedback is tied to the backend's message ID

Thumbs up/down buttons only render `if (isBot && msg.message_id && !feedback)` – the ID comes from the backend's database write (`db.save_message()` returns the auto-increment ID), so feedback can be attributed to one specific stored message (e.g. "bot reply," which would be ambiguous if messages arrived out of order or the array

6. Deep Dive 2 – Outage Board (OutageBoard.jsx)

Polling instead of a live connection

```
useEffect(() => {  
  load()  
  const id = setInterval(load, 60_000) // refresh every minute  
  return () => clearInterval(id)  
}, [])
```

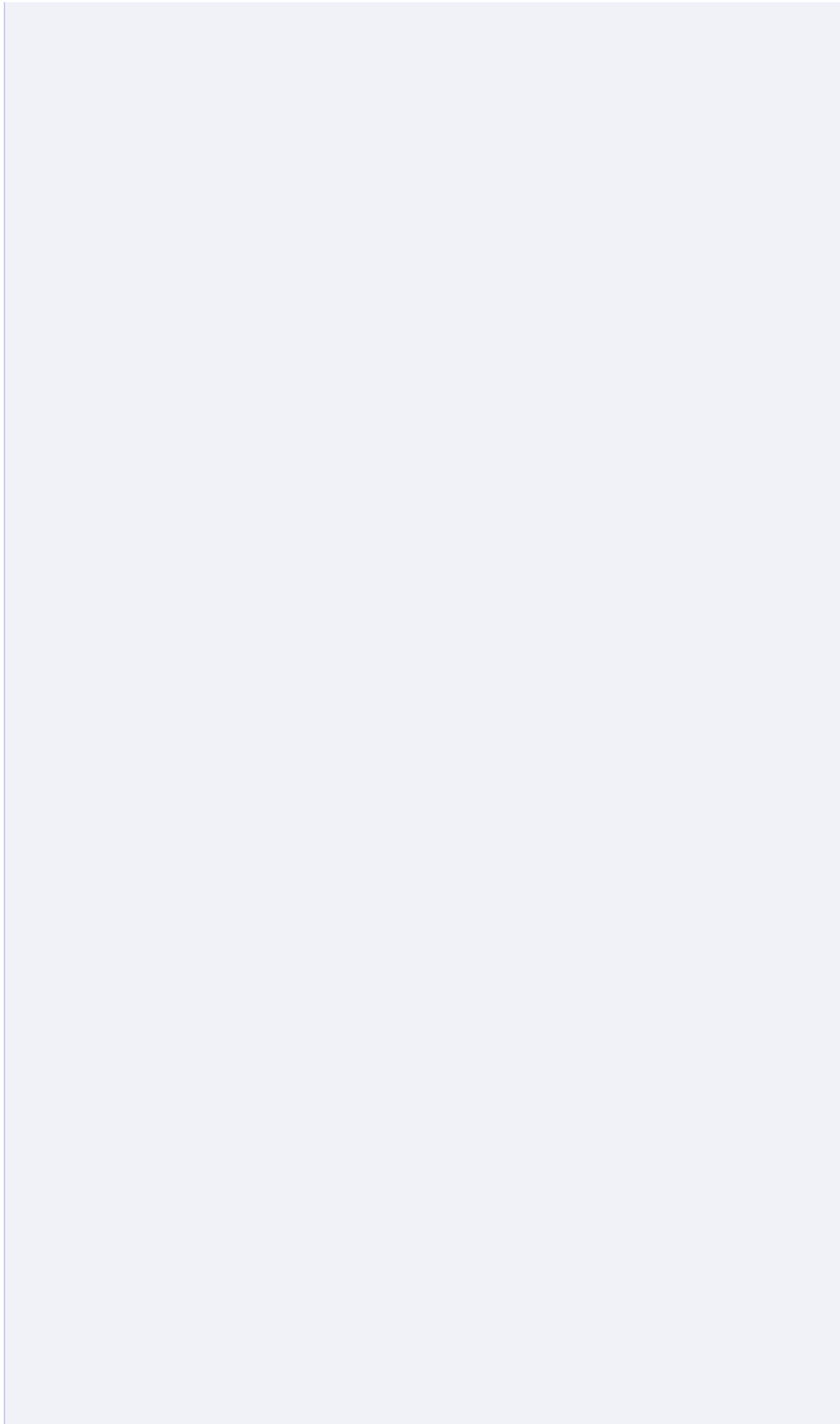
The outage board deliberately uses simple 60-second polling of `GET /outages` rather than WebSocket. Outage status changes on the scale of minutes-to-hours (the backend's status refreshes hourly – see the backend guide, Section 7), so a persistent connection for no meaningful gain in freshness. Polling is the right-sized tool here, in contrast to WebSocket which is justified by genuinely sub-second interaction.

Client-side filtering keeps the UI instantly responsive

The area search box and status filter buttons (`all` / `active` / `scheduled` / `resolved`) both filter the already-fetched `outages` array in-memory (`Array.filter`), rather than re-querying the backend per keystroke. Given the dataset size (a handful to a few dozen outage records), this avoids unnecessary network requests and gives filtering that feels instantaneous.

Defensive date formatting

`fmt(iso)` wraps `new Date(iso)` in a try/catch and falls back to showing the raw ISO string if parsing fails, and separately treats a missing `estimated_restoration_time` as `"TBD"` rather than showing "Invalid Date" – because the scraper (backend) can legitimately return `null` for restoration time when ECG's website hasn't published one yet.



7. Deep Dive 3 – Fault Report Wizard (FaultForm.js)

Why a 3-step wizard instead of one long form

Splitting into Customer Details → Fault Type & Urgency → Review & Submit reduces effort of the form (one short screen at a time) and lets each step validate independently so the user can proceed – critical for a stressed customer reporting a possibly dangerous fault. They don't have to scroll through a long form under pressure.

Per-step validation gating

```
const canNext = [  
  () => form.customer_name && form.phone && form.location,  
  () => form.fault_type && form.urgency,  
  () => true,  
]
```

Each step has its own validator function in an array indexed by the current step. The next step is disabled until `canNext[step]()` returns true. This keeps validation logic co-located with the step definitions rather than one large conditional, and makes it trivial to

The danger notice is derived from data already collected, not asked

Both the review screen and the post-submission success screen check `['critical', 'high'].includes(form.urgency)` to show a "call 0800-POWER now" message. Reusing the urgency value the customer already selected in Step 2 is reused to decide whether to show the visual warning, rather than adding a redundant "is this an emergency?" question.

Failure handling matches the safety stakes of the form

If `POST /fault-report` fails, the error message explicitly directs the customer to call 0800-POWER rather than just saying "try again" – because a failed fault report situation (fallen wire, sparking transformer) must never leave the customer with no guidance.

8. Deep Dive 4 – Admin Panel & Analytics Dashboard

Token storage: `sessionStorage`, not `localStorage`

```
sessionStorage.setItem(TOKEN_KEY, token)
```

The admin token is stored in `sessionStorage` deliberately: it is cleared automatically when the browser tab is closed, unlike `localStorage` which would persist indefinitely on a public machine. This narrows the window during which a stolen or forgotten session ID could be used.

One authenticated-fetch wrapper, used everywhere

```
export function adminFetch(url, options = {}) {
  const token = sessionStorage.getItem(TOKEN_KEY) || ''
  return fetch(url, {
    ...options,
    headers: { ...(options.headers || {}), Authorization: `Bearer ${token}` },
  })
}
```

Every admin tab (`FaultReportsTab`, `EscalationsTab`, `RagSourcesTab`) calls `adminFetch()` instead of the raw `fetch()` – this guarantees the Bearer token is attached consistently and means a future change to the auth header (e.g. scheme) only needs to be made in one place.

Each tab owns its own loading/error/data state

`FaultReportsTab`, `EscalationsTab`, and `RagSourcesTab` are structured

identically: local `loading` / `error` /data state, a `load()` function called on mount, and a manual "Retry"/"Refresh" button. This repetition is intentional across three tabs with near-identical but not fully identical rendering (fault reports coding, escalations have a status badge, RAG sources have nested source lists) demonstrating extracting a shared generic-list abstraction.

Analytics – hand-built visualisations instead of a charting library

`SatisfactionHistogram` and `IntentBar` are both implemented as plain `<div>` elements whose height/width is set via inline `style` as a percentage of the maximum value in the dataset:

```
const pct = max > 0 ? Math.round((count / max) * 100) : 0
...
<div className={`h-full ${meta.color} rounded-full`} style={{ width: `${pct}%`
```

This avoids adding a charting dependency (e.g. Chart.js, Recharts) for what is, and one histogram – both of which are simpler to hand-build with Tailwind than a general-purpose charting API.

9. Deep Dive 5 – App Shell, Routing & Network Con

Two independent entry points selected by URL path

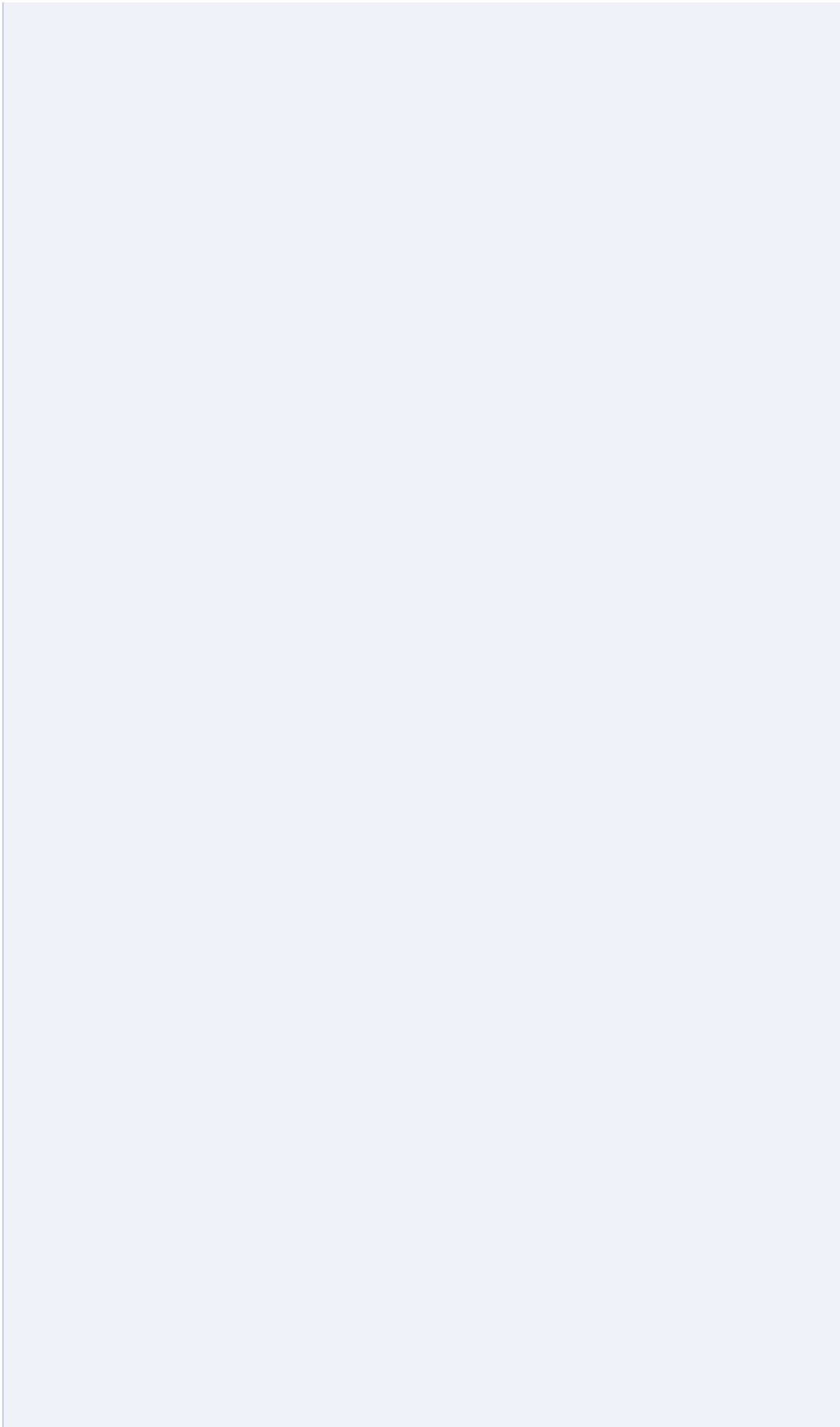
Both `main.jsx` and `App.jsx` independently check `window.location.pathname.startsWith('/admin')`. This is deliberately checked point (`main.jsx`) so the admin bundle path never renders any customer UI at all same check exists defensively inside `App.jsx` in case it is ever rendered direc

The proxy is what makes relative URLs possible

```
// vite.config.js
server: {
  host: true,          // listen on 0.0.0.0 – reachable from other devices on
  proxy: {
    '/ws': { target: 'ws://localhost:8000', ws: true, changeOrigin: true },
    '/api': { target: 'http://localhost:8000', rewrite: p => p.replace(/^\/api/, '') },
  }
}
```

```
// config.js
export const API = '/api'
export const WS_URL = (id) => `ws://${window.location.host}/ws/${id}`
```

Because every request goes through `/api` or `/ws` and is proxied server-side by Vite to `localhost:8000`, the frontend code never needs to know the backend's real address. `WS_URL` builds its target from `window.location.host` – whatever hostname or IP address the browser actually used to load the page – so opening the app at `http://192.168.1.20:5173` from a phone on the same Wi-Fi automatically connects WebSocket to `ws://192.168.1.20:5173/ws/...`, which Vite then proxies through to the backend. No environment variable or config change is needed to demo the app on another de



10. Design Trade-offs – Alternatives Considered

Decision point	Alternative(s) considered
Client-side routing	<code>react-router-dom</code>
Global state management	Redux / Context API / Zustand
Chat transport	REST polling for chat messages
Outage board data transport	WebSocket (matching chat)
Analytics charts	Chart.js / Recharts / D3
Admin token storage	<code>localStorage</code>
Styling approach	Component-level CSS files / CSS-in-JS (styled-c

11. Anticipated Defense Questions & Model Answers

Q. Why React and Vite specifically, and not Angular or plain JS?

A. The project's own Phase-1 comparison (speed, scalability, cost, usability) favored Vite+React: near-instant dev server reloads, a component model that scales cleanly, added, free and open-source, and a gentler learning curve than Angular's strict template. Plain HTML/CSS/JS was rejected because a multi-view app with shared UI patterns (forms, live data) becomes unmanageable without componentisation.

Q. Why is there no router library if there are multiple "pages"?

A. The three customer tabs and four admin tabs aren't real navigable routes – none need a shareable URL, deep-linking, or back-button semantics; they're just "which is shown," which a single `useState` handles. The only structural route split (customer vs admin) is resolved once at the entry point by reading `window.location.pathname`.

Q. How does the frontend stay secure when rendering AI-generated text as HTML?

A. Bot messages are rendered via `dangerouslySetInnerHTML` to support bold text and line breaks, but every message is first passed through `DOMPurify.sanitize()` with an explicit allow-list of six safe tags. Sanitisation runs last, after markdown-to-html transforms, so nothing those earlier steps introduce can bypass the filter.

Q. What happens if the WebSocket connection drops mid-chat?

A. The `onclose` handler schedules a reconnect with exponential backoff (1s, 2s, 4s, 8s, capped at 30s), shows a "Reconnecting..." indicator, and resets the backoff when a new connection succeeds – so a brief network blip recovers quickly, while a long outage keeps the server with reconnect attempts.

Q. How does the same frontend build work both on localhost and from another device?

A. The WebSocket URL is built at runtime from `window.location.host` rather than a hardcoded address, and the Vite dev server is configured with `host: true` to bind to `0.0.0.0`. Whatever hostname/IP a browser used to load the page is automatically used to reach the backend through the same Vite proxy, so no environment-specific configuration is needed to run the demo on a phone or another laptop on the same Wi-Fi.

Q. Why is the admin panel a completely separate component tree instead of a protected route?

A. Selecting between `App` and `AdminPanel` happens at the earliest possible point, in `main.jsx`, before any customer-facing component ever mounts. Admin-only UI and data never even exists in memory for a regular customer's session, so there's no risk of an admin element flashing on screen before an auth check resolves.

Q. How is the admin password protected on the frontend?

A. The password itself is never stored in frontend code or state beyond the login; it's sent once to `POST /admin/login`, which returns an opaque token. That token is stored in `sessionStorage` (cleared when the tab closes) and attached automatically to every admin request through a single `adminFetch()` wrapper.

Q. Why does the outage board poll every 60 seconds instead of using a live connection?

A. The backend's own scraper only refreshes live outage data once an hour, so a Websocket here would add complexity without any real gain in freshness. 60-second polling is a good compromise for how often the underlying data can actually change, in contrast to chat, where messages are genuinely needed.

Q. Why is filtering on the outage board done client-side instead of via API query parameters?

A. The dataset is small (a handful to a few dozen records fetched once per minute), so filtering the already-loaded array in memory gives instant, no-network-latency results. User clicks a status filter – adding server-side query parameters would only add network overhead with no benefit at this scale.

Q. Why build the analytics charts by hand instead of using a charting library like

A. Only two visual shapes are needed – a horizontal bar (intent distribution) and histogram (star ratings) – both trivially expressed as Tailwind-styled divs with width/height. A charting library would add a dependency and API surface disproportionate to the visualisations.

Q. How does per-message feedback (thumbs up/down) know which exact message it's rating?

A. Every bot reply the backend sends includes the database row ID it was saved under (`message_id`). The feedback buttons only render once that ID is present and post verbatim to `/message-feedback`, so a rating is always attributed to one specific message rather than inferred from message order.

Q. Why a 3-step wizard for the fault report form instead of one page?

A. Splitting details, fault type/urgency, and review into separate steps reduces page length, cognitive effort and allows per-step validation to gate progress – important for a customer reporting a dangerous situation under stress and shouldn't have to navigate one long, all-

Q. If this were to scale to many more users, what in the frontend would need to change?

A. Each admin tab currently fetches its own data independently with no caching or pagination – at high data volumes, `/admin/fault-reports` and similar endpoints would need pagination and the tabs would need to request pages rather than full lists. The components themselves are already per-session/stateless on the client and would not need changes.

12. Glossary of Key Terms

Term	Meaning in this project
SPA (Single-Page Application)	The entire customer app or admin panel
Vite dev-server proxy	Configuration that forwards <code>/api</code> and
Exponential backoff	A reconnect strategy where each retry
DOMPurify	A JavaScript library that strips unsafe
Session token (Bearer auth)	A random string issued by the backend
<code>sessionStorage</code> vs <code>localStorage</code>	Both are browser key-value stores; <code>s</code>
Optimistic/self-owned component state	Each tab component manages its own lo
Utility-first CSS (Tailwind)	A CSS approach where small, single-pu